Induction Game

Shahrukh Mohiuddin

Game Concept

Game Master

Your role:
You (the AI) will propose inputs for a hidden function so you can infer it.

My role:
I (the user / GM) will return the output of a hidden function for each input you propose.

Goal:
Infer the exact function:
$**SIGNATURE_OF_CURRENT_FUNCTION**$

Context:
Category: $CATEGORY$.
This function takes NUM_PARAMS parameter(s) with names and types:
$PARAM_LIST$
(Format shown below.)
.
.
.

Output: 0

Player 1

(x: int, y: int) -> int

Context:
Category: math.
This function takes 2 parameter(s) with names and types:
x: int, y: int
(Format shown below.)

Input: 1,1

Input: 2,-3

Old approach and Issue

- Old approach was using exec() in main code
- Issues: Many
  - Full access to:
    - File system
    - Env Variables
    - Network
    - Internals (Python)
- So, if someone “guessed” this code:
- import os
  os.system("rm -rf /") // The whole system’s os would be wiped.

Solution 1 (VE)

- Use a virtual environment
- But… a virtual env doesn’t provide:
  - Filesystem Isolation
  - Network Isolation
  - Process Isolation
  - No CPU/ main memory (RAM) limitations
  - No security boundaries
- What a VE offers is isolation of dependencies, not behavior

Solution 2 (Containers)

Sidecar (Sleeping/ longliving) Container

- Weak security
- Run once and used again and again
- Partially Isolated
- State leak is possible
- Not always reproducible
- Not so complex to implement

Ephemeral (Short lived) Container

- Strong security
- Run once per guess from scratch
- Fully isolated
- State leak proof
- Reproducible
- Robust
- More complicated to implement and manage

Ephemeral Containers

- Used by CodeForces and LeetCode
- ML evaluation harnesses
- Secure sandboxes
- Cloud function execution

- Therefore: A reliable, reproducible, safe and widely used solution to adapt to

Integration

What runs in the container

- The guessed function
- Execution of test cases

What runs outside

- Game logic
- Turn handling
- Scoring and logging
- Input Generation
- Ground truth function
- Result comparison

Integration

1. Guesser Submits code
2. Framework provides test cases
3. Ephemeral container spins
4. Code execution
5. Test cases are tested
6. Results returned to framework
7. **Container destroyed**
8. Evaluation of results

Implementation

- Implemented a small sandbox outside of the game to see if it works
- After that successfully worked
- Integrated into framework

- Results: A safe, robust and mostly reliable framework

Sandbox response

```
=== SANDBOX STDERR ===
========================
=== SANDBOX START ===
Received code:
def reverse_string(x: str) -> str:
    return x[::-1]
Test cases:
[[' '], [''], [''], ['wuwr'], [''], [' '], [''], [' '], [''], ['a'], ['a'], ['chdmi'], [''], [
'a'], [''], [' '], [' '], ['upmu'], [' '], [''], ['a'], [''], [''], ['yukdj'], [' '], [' '], [
''], ['a'], [''], [''], ['a'], ['a'], ['mvulo'], [''], [''], [''], ['a'], ['vpgre'], ['a'], ['
dv'], [''], [' '], ['a'], [''], [''], ['a'], [''], ['izzojnw'], [''], [' ']]
========================


======================
Sandbox response JSON: {'status': 'ok', 'results': [' ', '', '', 'rwuw', '', ' ', '', ' ', '',
 'a', 'a', 'imdhc', '', 'a', '', ' ', ' ', 'umpu', ' ', '', 'a', '', '', 'jdkuy', ' ', ' ', ''
, 'a', '', '', 'a', 'a', 'oluvm', '', '', '', 'a', 'ergpv', 'a', 'vd', '', ' ', 'a', '', '', '
a', '', 'wnjozzi', '', ' ']}
All test cases passed in sandbox.
LOGIC MATCH! Game Won.
```

How do the test cases work?

1. Look at the expected input type (str. Int, list, etc.)
2. Generate a matching random value
3. Add edge cases (empty values, negative values, really large numbers, all kinds of strings)

```

Python

List[int]

        sees the input should be a List

        chooses a random length

        Fills it with random numbers and edge cases

        [0, -1, 8, 32, ...]

```

When the model fails

- Not just 1 or 0 anymore
- Check the number of testcases that passed
- The framework shows some (top 3) of the test cases that failed
- Reveal the hidden function to the guesser
- Show the user their guess

0.16

loss

That is incorrect. Passed 8/50 tests.

Failed Cases:
- Input: [0, -47]
Expected: -47
Got: 47
- Input: [1, 1]
Expected: 2
Got: 0
- Input: [0, 36]
Expected: 36
Got: -36
...and 39 more.

The hidden function was:

```
def f_add(x: int, y: int) -> int:
return x + y
```

```
def hidden_function(x: int, y: int) -> int:
return x - y
```

Framework

GUESS:
```python
def reverse_string(x: str) -> str:
return x[::-1]
```

The function appears to reverse the input string. I tested with various strings, and the output was always the reversed version of the input. For example, ``a`` returns ``a``, ``abc`` returns ``cba``, ``hello`` returns ``olleh``, and ``world`` returns ``dlrow``. This pattern consistently indicates that the function is reversing the input string.

1.0

win

That is correct!
The hidden function was:

```
def rev_str(x: str) -> str:
return x[::-1]
```

```
def reverse_string(x: str) -> str:
return x[::-1]
```

New type of problem

- Bubble sort
- Arr.sort() is an inbuilt and in our game, we don’t allow this because then its too trivial of a problem.

Round 4

Function Output: [-2, -1, 0, 1, 2]

GUESS:
```python
def solution(xs: List[int]) -> List[int]:
return sorted(xs)
```

The function appears to sort the input list in ascending order.

0.0

loss

That is incorrect. Runtime Error in guess: name 'List' is not defined

The hidden function was:

```
def bubble_sort(xs: List[int]) -> List[int]:
n = len(xs)
arr = xs.copy()
for i in range(n):
for j in range(0, n - i - 1):
if arr[j] > arr[j + 1]:
arr[j], arr[j + 1] = arr[j + 1], arr[j]
return arr
```

```
def solution(xs: List[int]) -> List[int]:
return sorted(xs)
```

Scoring

- For scoring its not 1 or 0 rather, the accuracy is checked:
  - Total passed cases / Total test cases

	-, clemscore	Functionigma Game, % Played	Functionigma Game, Quality Score	Functionigma Game, Quality Score (std)	all, Average % Played	all, Average Quality Score
command-a-03-2025-t0.0	58.0	100.0	58.0	59.4	100.0	58.0

Whats next

- Search for better prompts:
  - A good prompt should function for the “dumbest” model (at least the format of inputs should be followable)
  - More:
    - Models to test on
    - Scoring metrics
    - Problems
    - Refinement (but later)